

SRS si SDD – PCD T17 Semantic Analyzer

Echipa proiect PCD

2026

Cuprins

1	Introducere	2
2	Scopul proiectului	2
3	Descriere generala	2
4	Utilizatori	2
4.1	Utilizator normal	3
4.2	Administrator	3
5	Cerinte functionale	3
5.1	FR1 – Pornirea serverului	3
5.2	FR2 – Acceptarea conexiunilor	3
5.3	FR3 – Concurenta	3
5.4	FR4 – Autentificare	3
5.5	FR5 – Upload fisier	3
5.6	FR6 – Salvare fisier	3
5.7	FR7 – Analiza semantica	4
5.8	FR8 – Generare raport	4
5.9	FR9 – Returnare rezultat	4
5.10	FR10 – Download raport	4
5.11	FR11 – Client admin	4
5.12	FR12 – Client Python	4
5.13	FR13 – Logging	4
6	Cerinte nefunctionale	4
6.1	NFR1 – Sistem de operare	4
6.2	NFR2 – Compilare stricta	4
6.3	NFR3 – Modularitate	5
6.4	NFR4 – Simplitate	5
6.5	NFR5 – Biblioteca externa	5
7	Arhitectura sistemului	5

8	Design detaliat	5
8.1	Server	5
8.2	Client normal	6
8.3	Client admin	6
8.4	Analyzer	6
8.5	Client Python	7
9	Fluxuri principale	7
9.1	Flux upload si analiza	7
9.2	Flux admin stats	7
9.3	Flux download raport	7
10	Structuri de date si fisiere	8
10.1	Fisiere de configurare	8
10.2	Foldere generate	8
11	Testare	8
11.1	Compilare	8
11.2	Test fisier valid	8
11.3	Test fisier cu eroare de tip	8
11.4	Test fisier cu warning	9
11.5	Test concurenta	9
11.6	Test client Python	9
11.7	Test admin	9
12	Impartirea muncii in echipa	9
13	Limitari	10
14	Posibile extensii	10
15	Concluzie	10

1 Introducere

Acest document contine specificatia cerintelor software (SRS) si descrierea designului software (SDD) pentru proiectul PCD T17 – Semantic Analyzer for Source Code.

Proiectul consta intr-o aplicatie client-server care permite analiza semantica a fisierelor sursa C. Clientul trimite fisierul catre server, serverul il salveaza, ruleaza un analyzer bazat pe `libclang` si trimite inapoi raportul rezultat.

2 Scopul proiectului

Scopul proiectului este realizarea unei aplicatii distribuite care combina:

- programarea cu socket-uri TCP;
- procese concurente;
- comunicarea client-server;
- autentificarea utilizatorilor;
- logarea operatiilor;
- analiza semantica de cod sursa;
- folosirea unei biblioteci externe.

Tema aleasa este T17 – Analiza Semantica Cod Sursa.

3 Descriere generala

Aplicatia permite utilizatorilor sa trimita fisiere C catre server. Serverul analizeaza fisierul si genereaza un raport text. Raportul contine informatii despre diagnostice, functii, variabile si structuri de control.

Sistemul include:

- server TCP in C;
- client normal in C;
- client admin in C;
- client alternativ in Python;
- modul analyzer in C;
- fisiere de configurare;
- loguri si rapoarte.

4 Utilizatori

Sistemul are doua tipuri principale de utilizatori:

4.1 Utilizator normal

Un utilizator normal poate:

- sa se autentifice;
- sa trimita fisiere sursa pentru analiza;
- sa primeasca rezultatul analizei;
- sa descarce rapoarte.

4.2 Administrator

Un administrator poate:

- sa vada statisticile serverului;
- sa vada logurile serverului;
- sa listeze fisierele uploadate;
- sa listeze rapoartele generate.

5 Cerinte functionale

5.1 FR1 – Pornirea serverului

Serverul trebuie sa porneasca pe adresa 127.0.0.1 si portul 8080.

5.2 FR2 – Acceptarea conexiunilor

Serverul trebuie sa accepte conexiuni TCP de la clienti.

5.3 FR3 – Concurenta

Serverul trebuie sa poata gestiona mai multi clienti prin procese separate.

5.4 FR4 – Autentificare

Serverul trebuie sa autentifice utilizatorii folosind fisierul:

```
config/users.cfg
```

5.5 FR5 – Upload fisier

Clientul normal trebuie sa poata trimite un fisier sursa C catre server.

5.6 FR6 – Salvare fisier

Serverul trebuie sa salveze fisierele primite in folderul:

```
uploads/
```

5.7 FR7 – Analiza semantica

Serverul trebuie sa ruleze analyzer-ul pentru fisierul primit.

5.8 FR8 – Generare raport

Serverul trebuie sa genereze un raport text in folderul:

```
reports/
```

5.9 FR9 – Returnare rezultat

Serverul trebuie sa trimita rezultatul analizei catre client.

5.10 FR10 – Download raport

Clientul trebuie sa poata descarca un raport generat anterior.

5.11 FR11 – Client admin

Clientul admin trebuie sa poata cere statistici, loguri, lista fisierelor uploadate si lista rapoartelor.

5.12 FR12 – Client Python

Sistemul trebuie sa includa un client intr-un alt limbaj, in cazul acesta Python.

5.13 FR13 – Logging

Serverul trebuie sa scrie actiunile importante in:

```
logs/server.log
```

6 Cerinte nefunctionale

6.1 NFR1 – Sistem de operare

Aplicatia este destinata rularii pe Linux sau WSL.

6.2 NFR2 – Compilare stricta

Codul trebuie sa compileze fara warning-uri cu:

```
-Wall -Wextra -Werror -pedantic -std=c11
```

6.3 NFR3 – Modularitate

Codul trebuie sa fie impartit in module clare:

- server;
- client;
- admin client;
- analyzer;
- client Python.

6.4 NFR4 – Simplitate

Protocolul de comunicare este text-based si usor de testat.

6.5 NFR5 – Biblioteca externa

Aplicatia foloseste libclang pentru analiza codului sursa.

7 Arhitectura sistemului

Arhitectura generala este:

```
+-----+ TCP +-----+
| Client C | -----> | |
+-----+ | |
| Server | ---> Analyzer libclang
+-----+ TCP | |
| Admin C | -----> | |
+-----+ +-----+
|
+-----+ TCP |
| Python | -----+
+-----+
```

Serverul reprezinta componenta centrala. Toate cererile ajung la server, iar serverul decide ce operatie trebuie executata.

8 Design detaliat

8.1 Server

Serverul are urmatoarele responsabilitati:

- creeaza socket TCP;
- face bind;
- face listen;

- accepta clienti;
- creeaza procese copil pentru clienti;
- autentifica utilizatorii;
- primeste fisiere;
- ruleaza analyzer-ul;
- trimite rezultate;
- scrie loguri.

8.2 Client normal

Clientul normal are urmatoarele responsabilitati:

- se conecteaza la server;
- se autentifica;
- trimite fisierul sursa;
- primeste rezultatul analizei;
- afiseaza rezultatul in terminal.

8.3 Client admin

Clientul admin ofera un meniu simplu:

```
1. Server stats
2. Show server logs
3. List uploaded files
4. List reports
5. Quit
```

8.4 Analyzer

Analyzer-ul este responsabil cu analiza codului sursa. Acesta foloseste `libclang` pentru parsarea fisierului C.

Analyzer-ul extrage:

- diagnostice;
- numar de functii;
- numar de variabile;
- numar de instructiuni `if`;
- numar de bucle `for`;
- numar de bucle `while`.

8.5 Client Python

Clientul Python ofera doua operatii:

- upload;
- download.

Exemple:

```
python3 python_client/client.py upload tests/sample.c
python3 python_client/client.py download sample_report.txt
```

9 Fluxuri principale

9.1 Flux upload si analiza

1. Clientul se conecteaza la server.
2. Clientul se autentifica.
3. Clientul trimite fisierul sursa.
4. Serverul salveaza fisierul in **uploads/**.
5. Serverul ruleaza analyzer-ul.
6. Analyzer-ul genereaza raportul.
7. Serverul salveaza raportul in **reports/**.
8. Serverul trimite raportul catre client.

9.2 Flux admin stats

1. Adminul porneste clientul admin.
2. Adminul alege optiunea **Server stats**.
3. Clientul trimite cererea catre server.
4. Serverul citeste statisticile.
5. Serverul trimite raspunsul catre client.

9.3 Flux download raport

1. Clientul cere un raport dupa nume.
2. Serverul cauta raportul in **reports/**.
3. Serverul trimite continutul raportului.
4. Clientul salveaza raportul local.

10 Structuri de date si fisiere

10.1 Fisiere de configurare

Fisier	Rol
config/app.cfg	configurare generala aplicatie
config/users.cfg	utilizatori si roluri

10.2 Foldere generate

Folder	Rol
uploads/	fisiere primite de la clienti
reports/	rapoarte generate
downloads/	rapoarte descarcate de clientul C
downloads_py/	rapoarte descarcate de clientul Python
logs/	loguri si statistici server

11 Testare

11.1 Compilare

```
make clean  
make
```

Rezultatul asteptat este compilarea fara warning-uri.

11.2 Test fisier valid

```
./client tests/sample.c
```

Rezultatul asteptat:

- diagnostice 0;
- raport generat;
- rezultat afisat clientului.

11.3 Test fisier cu eroare de tip

```
./client tests/bad_type.c
```

Rezultatul asteptat:

- diagnostic de tip error;
- raport generat;
- eroarea afisata in rezultat.

11.4 Test fisier cu warning

```
./client tests/bad_uninitialized.c
```

Rezultatul asteptat:

- diagnostic de tip warning;
- raport generat.

11.5 Test concurenta

```
./client tests/sample.c & ./client tests/bad_type.c & wait
```

Rezultatul asteptat:

- ambele cereri sunt procesate;
- ambele rapoarte sunt generate;
- serverul continua sa ruleze.

11.6 Test client Python

```
python3 python_client/client.py upload tests/good_while.c  
python3 python_client/client.py download good_while_report.txt
```

11.7 Test admin

```
./admin_client
```

Se testeaza:

- stats;
- logs;
- list uploaded files;
- list reports.

12 Impartirea muncii in echipa

Echipa are 5 membri. O impartire posibila a responsabilitatilor este:

Persoana	Responsabilitate
Persoana 1	server TCP si gestionarea conexiunilor
Persoana 2	client normal si protocol de comunicare
Persoana 3	analyzer libclang si generare rapoarte
Persoana 4	admin client, loguri si statistici

Persoana 5	client Python, teste si documentatie
------------	--------------------------------------

13 Limitari

Aplicatia este realizata pentru scop educational si ruleaza local. Limitari:

- nu foloseste TLS;
- autentificarea este simpla, bazata pe fisier;
- nu foloseste baza de date;
- suportul este orientat spre fisiere C;
- analiza este limitata la metricile implementate.

14 Posibile extensii

Aplicatia poate fi extinsa prin:

- suport mai bun pentru C++;
- calcularea complexitatii ciclomatice;
- afisare HTML pentru rapoarte;
- interfata web;
- stocare in baza de date;
- suport pentru mai multe limbaje de programare.

15 Concluzie

Proiectul implementeaza o aplicatie client-server functionala pentru analiza semantica a codului sursa. Sistemul include server TCP concurent, client C, client admin, client Python, autentificare, loguri, rapoarte si analiza bazata pe `libclang`. Functionalitatile principale au fost testate pe mai multe fisiere sursa, atat valide, cat si cu erori.